Guided Exercise 2b

Test Cases Documentation

register_document Method

Course	Software Development – 2026
Module	Test-Driven Development
Degree	Computer Science & Engineering
Group	G87.2026.T12.GE2b
Date	April 2026

1. Introduction

This document presents the functional and structural test cases designed for the `register_document` method of the `EnterpriseManager` class, developed as part of Guided Exercise 2b (GE2b) for the Software Development course. The exercise follows the Test-Driven Development (TDD) methodology, where test cases are defined before implementation, and code is added incrementally until all tests pass.

The `register_document` method receives a file path pointing to a JSON file that must contain a **PROJECT_ID** (32 hexadecimal characters) and a **FILENAME** (8 alphanumeric characters followed by `.pdf`, `.docx`, or `.xlsx`). If validation succeeds, it creates a `ProjectDocument` instance, appends it to `all_documents.json`, and returns the SHA-256 file signature.

Method signature:

```
register_document(input_file: str) -> str
```

1.1 Method Summary

Aspect	Detail
Input	Path to a JSON file containing PROJECT_ID and FILENAME
Validation	File existence → JSON format → structure → value format
Processing	Instantiate ProjectDocument, compute SHA-256 signature, append to all_documents.json
Output (OK)	64-character hexadecimal SHA-256 string
Output (Error)	EnterpriseManagementException with descriptive message

2. Syntactic Analysis (Functional Testing)

The Syntactic Analysis technique derives test cases by modelling the input as a context-free grammar and then systematically applying mutations (deletion, duplication, modification) to each terminal and non-terminal node. This guarantees that every structural rule of the input format is exercised.

2.1 Grammar Definition

The input to `register_document` is a JSON file. The grammar below captures all structural and value constraints:

```
<FILE> ::= <OBJECT>
<OBJECT> ::= '{' <PAIR_LIST> '}'
<PAIR_LIST> ::= <PAIR_PID> ',' <PAIR_FN>
<PAIR_PID> ::= <KEY_PID> ':' <PID_VALUE>
<PAIR_FN> ::= <KEY_FN> ':' <FN_VALUE>
<KEY_PID> ::= "PROJECT_ID"
<KEY_FN> ::= "FILENAME"
<PID_VALUE> ::= <HEX32>
<FN_VALUE> ::= <NAME8> '.' <EXTENSION>
<HEX32> ::= [0-9a-f]{32} /* 32 lowercase hex chars */
<NAME8> ::= [a-zA-Z0-9]{8} /* 8 alphanumeric chars */
<EXTENSION> ::= 'pdf' | 'docx' | 'xlsx'
```

2.2 Grammar Node Index

Node	Symbol	T/NT	Description
1	FILE	NT	Root: the input file itself
2	OBJECT	NT	JSON object { … }
3	{	T	Opening brace (literal)
4	PAIR_LIST	NT	Sequence of two key-value pairs
5	}	T	Closing brace (literal)
6	PAIR_PID	NT	First key-value pair (PROJECT_ID)
7	,	T	Comma separator between pairs (literal)
8	PAIR_FN	NT	Second key-value pair (FILENAME)
9	"PROJECT_ID"	T	PROJECT_ID key string (literal)
10	:	T	Colon after PROJECT_ID key (literal)
11	PID_VALUE	NT	PROJECT_ID value node
12	"FILENAME"	T	FILENAME key string (literal)
13	:	T	Colon after FILENAME key (literal)
14	FN_VALUE	NT	FILENAME value node (NAME + . + EXTENSION)
15	HEX32	NT	Non-terminal wrapper for the 32-hex sequence
16	NAME8	NT	Non-terminal wrapper for the 8-alphanumeric sequence

Node	Symbol	T/NT	Description
17	[0-9a-f]{32}	T	Exactly 32 lowercase hexadecimal characters
18	.	T	Dot separator between name and extension (literal)
19	EXTENSION	NT	File extension sub-rule
20	[a-zA-Z0-9]{8}	T	Exactly 8 alphanumeric characters (file name)
21	pdf | docx | xlsx	T	One of three allowed extensions

2.3 Derivation Tree

The derivation tree below shows how a valid input is produced from the grammar start symbol `FILE`. Each level of the derivation is labelled **N1**–**N7** on the left margin; within each level the numbered blue squares identify the nodes introduced in §2.2, and the descriptive text above each square names the grammar symbol at that position. The bordered box below the tree shows the concrete input string that this derivation produces.

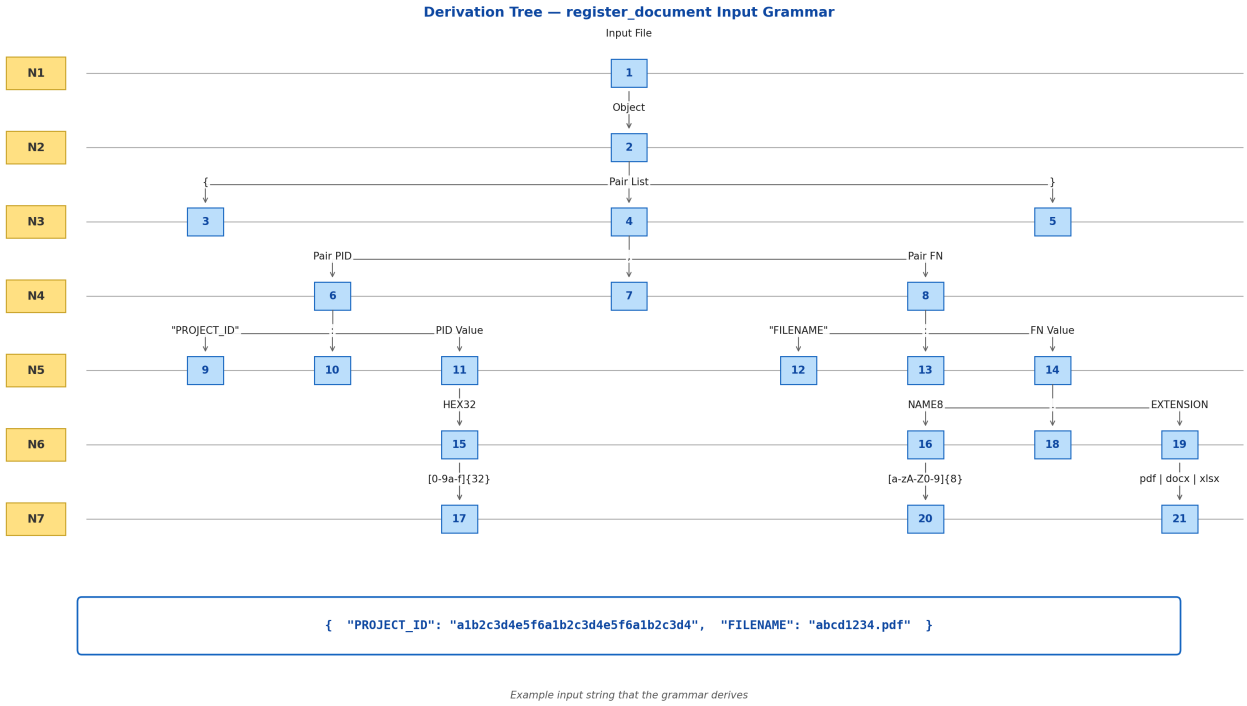

Figure 1 – Derivation tree for a valid register_document JSON input

2.4 Syntactic Analysis Test Cases

The 27 test cases below are derived by applying VALID, DELETION, DUPLICATION, and MODIFICATION mutations to each node in the derivation tree. Green rows are valid cases; orange rows are invalid/error cases.

#	Node	T/NT	Type	ID	Test File	Expected Result	Observations
1	1-21	NT+T	VALID	TC_M2_01	test_valid.json	SHA-256 hex string (64 chars) returned; all_documents.json updated	Baseline; all nodes covered
2	21	T	VALID	TC_M2_02	test_valid_docx.json	SHA-256 hex string (64 chars) returned; all_documents.json updated	Node 21 variant: .docx
3	21	T	VALID	TC_M2_03	test_valid_xlsx.json	SHA-256 hex string (64 chars) returned; all_documents.json updated	Node 21 variant: .xlsx
4	1	NT	DELETION	TC_M2_04	nonexistent.json	EnterpriseMgmtException: "Input file not found"	File absent from disk
5	2	NT	MODIFICATION	TC_M2_05	test_notjson.json	EnterpriseMgmtException: "File is not JSON formatted"	Node 2 replaced with plain text
6	4	NT	DELETION	TC_M2_06	test_empty.json	EnterpriseMgmtException: "JSON does not have the expected structure"	PAIR_LIST deleted
7	6	NT	DELETION	TC_M2_07	test_no_pid.json	EnterpriseMgmtException: "JSON does not have the expected structure"	PAIR_PID deleted
8	8	NT	DELETION	TC_M2_08	test_no_fn.json	EnterpriseMgmtException: "JSON does not have the expected structure"	PAIR_FN deleted
9	11	NT	DELETION	TC_M2_09	test_empty_pid.json	EnterpriseMgmtException: "JSON data has no valid values"	PID_VALUE empty string
10	14	NT	DELETION	TC_M2_10	test_empty_fn.json	EnterpriseMgmtException: "JSON data has no valid values"	FN_VALUE empty string
11	3	T	DELETION	TC_M2_11	test_no_open.json	EnterpriseMgmtException: "File is not JSON formatted"	{ deleted → syntax error

#	Node	T/NT	Type	ID	Test File	Expected Result	Observations
12	5	T	DELETION	TC_M2_12	test_no_close.json	EnterpriseMgmtException: "File is not JSON formatted"	} deleted → syntax error
13	7	T	DELETION	TC_M2_13	test_no_comma.json	EnterpriseMgmtException: "File is not JSON formatted"	, deleted → syntax error
14	10	T	DELETION	TC_M2_14	test_no_colon1.json	EnterpriseMgmtException: "File is not JSON formatted"	: after PROJECT_ID deleted
15	13	T	DELETION	TC_M2_15	test_no_colon2.json	EnterpriseMgmtException: "File is not JSON formatted"	: after FILENAME deleted
16	3	T	DUPLICATION	TC_M2_16	test_double_open.json	EnterpriseMgmtException: "File is not JSON formatted"	{ duplicated → syntax error
17	7	T	DUPLICATION	TC_M2_17	test_extra_comma.json	EnterpriseMgmtException: "File is not JSON formatted"	,, duplicated → syntax error
18	9	T	MODIFICATION	TC_M2_18	test_bad_key1.json	EnterpriseMgmtException: "JSON does not have the expected structure"	Key renamed to PROJECT
19	12	T	MODIFICATION	TC_M2_19	test_bad_key2.json	EnterpriseMgmtException: "JSON does not have the expected structure"	Key renamed to FILE
20	17	T	MODIFICATION	TC_M2_20	test_short_pid.json	EnterpriseMgmtException: "JSON data has no valid values"	31 hex chars (short by 1)
21	17	T	MODIFICATION	TC_M2_21	test_long_pid.json	EnterpriseMgmtException: "JSON data has no valid values"	33 hex chars (long by 1)
22	17	T	MODIFICATION	TC_M2_22	test_nonhex_pid.json	EnterpriseMgmtException: "JSON data has no valid values"	"z" is not a hex digit
23	20	T	MODIFICATION	TC_M2_23	test_short_name.json	EnterpriseMgmtException: "JSON data has no valid values"	NAME = 5 chars (short by 3)
24	20	T	MODIFICATION	TC_M2_24	test_long_name.json	EnterpriseMgmtException: "JSON data has no valid values"	NAME = 10 chars (long by 2)

#	Node	T/NT	Type	ID	Test File	Expected Result	Observations
25	20	T	MODIFICATION	TC_M2_25	test_special_name.json	EnterpriseMgmtException: "JSON data has no valid values"	"@" not alphanumeric
26	19	NT	DELETION	TC_M2_26	test_no_ext.json	EnterpriseMgmtException: "JSON data has no valid values"	EXTENSION node deleted
27	21	T	MODIFICATION	TC_M2_27	test_bad_ext.json	EnterpriseMgmtException: "JSON data has no valid values"	.txt not in allowed set

Legend: *VALID – input accepted* *DELETION – node removed* *DUPLICATION – node repeated* *MODIFICATION – node value changed*

3. Structural Testing

Structural (white-box) testing exercises the internal control flow of the method. A Control Flow Graph (CFG) is constructed from the implementation, the cyclomatic complexity is calculated, and a minimal set of linearly independent (basis) paths is identified to achieve full branch coverage.

3.1 CFG Node Definitions

The following 15 nodes represent the sequential and branching statements within `register_document`. Node numbering preserves 15 and 16 as the two `except` branches of the `try` block at node 12.

Node	Type	Statement / Description
1	Process	Entry point – receive input_file parameter
2	Decision	os.path.exists(input_file) — does the file exist on disk?
3	Process	raise EnterpriseManagementException("Input file not found")
4	Process	open(input_file) and json.load(file) — may raise JSONDecodeError
5	Process	raise EnterpriseManagementException("File is not JSON formatted")
6	Decision	isinstance(data, dict) AND "PROJECT_ID" in data AND "FILENAME" in data
7	Process	raise EnterpriseManagementException("JSON does not have the expected structure")
8	Decision	data["PROJECT_ID"] and data["FILENAME"] are non-empty
9	Process	raise EnterpriseManagementException("JSON data has no valid values") [empty values]
10	Decision	Regex match: PROJECT_ID = [0-9a-f]{32} AND FILENAME = [a-zA-Z0-9]{8}\.(pdf|docx|xlsx)
11	Process	raise EnterpriseManagementException("JSON data has no valid values") [format error]
12	Process	try: ProjectDocument(PROJECT_ID, FILENAME) + _save_document(document)
13	Process	return document.file_signature — SUCCESS
15	Process	except EnterpriseManagementException → re-raise as-is
16	Process	except Exception → raise EnterpriseManagementException("Internal processing error when getting the file_signature")

3.2 Control Flow Graph

The CFG below maps every decision and process node. Blue rectangles are process steps; orange diamonds are binary decisions (Y/N branches); red nodes raise exceptions; the green node represents the successful return.

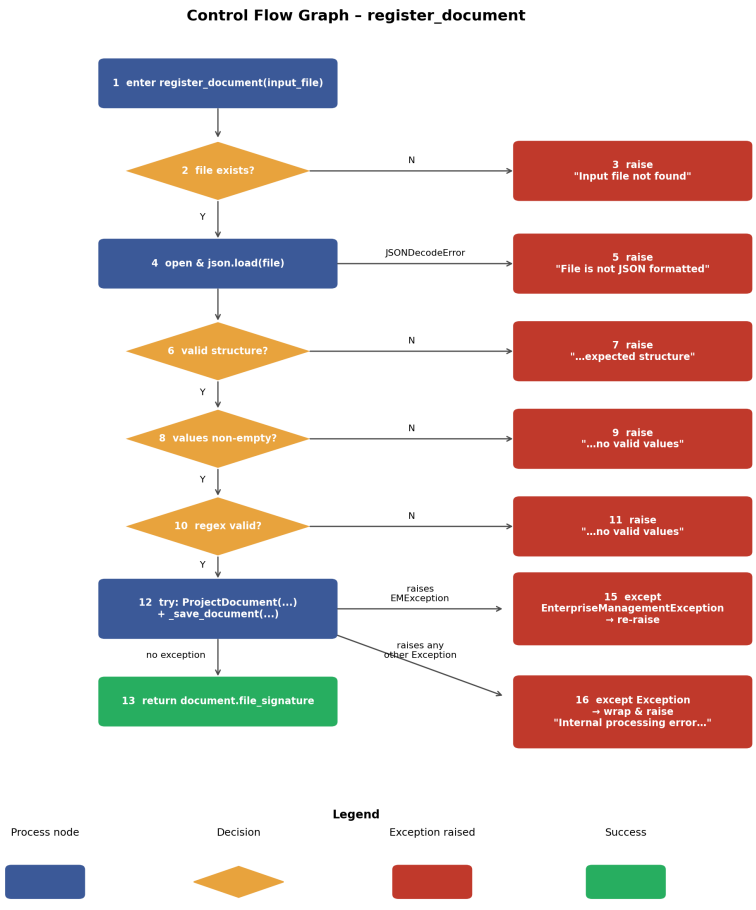

Figure 2 – Control Flow Graph of register_document

Figure 2 – Control Flow Graph of register_document

3.3 Cyclomatic Complexity

Cyclomatic complexity V(G) is calculated using McCabe's formula:

V(G) = E – **N + 2P**, where E = edges, N = nodes, P = connected components.

The graph enumerated in §3.1 contains **N = 15** nodes and **E = 14** internal edges. To apply McCabe's formula the CFG must be strongly connected, which is achieved by adding a virtual exit node joined from each of the 8 terminal nodes (3, 5, 7, 9, 11, 13, 15, 16). This yields N' = 16 and E' = 22:

V(G) = 22 – **16 + 2 × 1 = 8**

This can be cross-checked via the equivalent decision-count formula **V(G) = D + 1**, where D counts binary decisions. `register_document` contains 5 explicit binary decisions (nodes 2, 6, 8, 10, plus the implicit `JSONDecodeError` branch at node 4) and the three-way `try` / `except EnterpriseManagementException` / `except Exception` at node 12, which decomposes into 2 additional binary decisions. Therefore D = 5 + 2 = 7, giving V(G) = 7 + 1 = **8**.

Eight linearly independent basis paths are therefore required for full branch coverage, defined in §3.4.

3.4 Basis Path Definitions

The table below defines each basis path, the condition that drives execution along that path, and the expected outcome:

Path	Node Sequence	Condition	Expected Outcome
P1	1 → 2 → 3	file does not exist on disk	EnterpriseMgmtException: "Input file not found"
P2	1 → 2 → 4 → 5	file exists but content cannot be parsed as JSON	EnterpriseMgmtException: "File is not JSON formatted"
P3	1 → 2 → 4 → 6 → 7	JSON parsed OK; dict structure invalid or key missing	EnterpriseMgmtException: "JSON does not have the expected structure"
P4	1 → 2 → 4 → 6 → 8 → 9	structure valid; PROJECT_ID or FILENAME value is empty string	EnterpriseMgmtException: "JSON data has no valid values"
P5	1 → 2 → 4 → 6 → 8 → 10 → 11	values non-empty; PROJECT_ID or FILENAME fails regex validation	EnterpriseMgmtException: "JSON data has no valid values"
P6	1 → 2 → 4 → 6 → 8 → 10 → 12 → 13	all validations pass; ProjectDocument created and saved successfully	SHA-256 hex signature (64 chars) returned; all_documents.json updated
P7	1 → 2 → 4 → 6 → 8 → 10 → 12 → 15	try block raises EnterpriseManagementException internally (re-raise)	EnterpriseManagementException propagated unchanged
P8	1 → 2 → 4 → 6 → 8 → 10 → 12 → 16	try block raises any other Exception (e.g. IOError on file write)	EnterpriseMgmtException: "Internal processing error when getting the file_signature"

3.5 Loop Analysis

`register_document` contains no explicit `for` or `while` iterative constructs. However, two forms of implicit iteration are present and must be exercised at their boundary conditions:

(1) Persistent file state. The `_save_document` helper reads the existing `all_documents.json` array (if any) and appends a new entry. Each invocation therefore depends on and modifies the size-*N* state of the file, and classical 0 / 1 / *N* iteration-boundary cases apply to this array length.

(2) Regex engine iteration. The PROJECT_ID and FILENAME regex validations at node 10 iterate internally over every character of the input. Boundary-length inputs (0 chars, exact length, exact length ± 1) are already exercised by syntactic test cases TC_M2_09, TC_M2_10, TC_M2_20, TC_M2_21, TC_M2_23, and TC_M2_24, so they are not duplicated here.

The three file-state boundary cases are therefore:

Case	Description	Pre-condition	Expected Result
L1 — zero iterations	all_documents.json does not yet exist	Delete the file before calling register_document	File created with a single-entry array; valid SHA-256 signature returned
L2 — one existing entry	all_documents.json contains exactly one document	Register one document first, then register a second	File contains two entries; second signature returned
L3 — multiple existing entries	all_documents.json contains n > 1 entries	Register n documents, then register one more	File contains n+1 entries; latest signature returned

3.6 Structural Test Cases

The 8 structural test cases below exercise all basis paths P1–P8 identified in §3.4.

ID	Test Path	Input / Condition	Expected Result	Observations
TC_M2_ST_01	1 → 2 → 3	input_file = "nonexistent.json" (absent)	EnterpriseMgmtException: "Input file not found"	P1 – decision node 2 = False
TC_M2_ST_02	1 → 2 → 4 → 5	File content: "this is not json at all"	EnterpriseMgmtException: "File is not JSON formatted"	P2 – json.JSONDecodeError at node 4
TC_M2_ST_03	1 → 2 → 4 → 6 → 7	File: {"FILENAME":"abcd1234.pdf"} (PROJECT_ID key missing)	EnterpriseMgmtException: "JSON does not have the expected structure"	P3 – decision node 6 = False
TC_M2_ST_04	1 → 2 → 4 → 6 → 8 → 9	File: {"PROJECT_ID":"","FILENAME":"abcd1234.pdf"}	EnterpriseMgmtException: "JSON data has no valid values"	P4 – decision node 8 = False (empty PID)

ID	Test Path	Input / Condition	Expected Result	Observations
TC_M2_ST_05	1 → 2 → 4 → 6 → 8 → 10 → 11	File: {"PROJECT_ID":"z1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4","FILENAME":"abcd1234.pdf"}	EnterpriseMgmtException: "JSON data has no valid values"	P5 – regex at node 10 fails (leading "z" is non-hex)
TC_M2_ST_06	1 → 2 → 4 → 6 → 8 → 10 → 12 → 13	File: {"PROJECT_ID":"a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4","FILENAME":"abcd1234.pdf"}	64-char SHA-256 string; all_documents.json updated	P6 – happy path (all decisions pass)
TC_M2_ST_07	1 → 2 → 4 → 6 → 8 → 10 → 12 → 15	Valid JSON; patch ProjectDocument.__init__ to raise EnterpriseManagementException("internal pre-check failed")	EnterpriseManagementException propagated unchanged (original message preserved)	P7 – except EnterpriseManagementException re-raise branch (node 15)
TC_M2_ST_08	1 → 2 → 4 → 6 → 8 → 10 → 12 → 16	Valid JSON; patch _save_document to raise IOError	EnterpriseMgmtException: "Internal processing error when getting the file_signature"	P8 – generic except Exception branch (node 16)

4. Test Coverage Summary

The table below summarises total test cases and coverage achieved across both testing techniques applied to `register_document`:

Technique	Test Cases	Nodes / Paths Covered	Coverage
Syntactic Analysis	27	All 21 grammar nodes[1]	100% node coverage
Structural Testing	8	All 8 basis paths (P1–P8); all CFG branches	Full branch coverage (V(G) = 8)
Loop Testing	3	Empty, single-entry, multi-entry array states	100% loop boundary coverage
Total	38	—	—

[1] Nodes 15, 16, and 18 are intermediate wrappers (HEX32, NAME8, and the dot separator). Their behaviour is exercised transitively through their child terminal nodes 17, 20, and 21, which appear as explicit targets in the syntactic test cases.

5. References

1. UC3M Software Development Course – *Guided Exercise 2b Statement* (February 2026).

2. Myers, G. J., Badgett, T., Sandler, C. – *The Art of Software Testing*, 3rd ed., Wiley, 2011.

3. McCabe, T. J. – "A Complexity Measure", *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, 1976.

4. Python Documentation – `hashlib` module: https://docs.python.org/3/library/hashlib.html